

A VISUAL WRITE-UP

Nanite & Lumen

Virtualized micro-polygon geometry and
fully dynamic global illumination in Unreal Engine
5

Part I Nanite

How to draw billions of source triangles at roughly one triangle per pixel: a cluster DAG built by repeated group-simplify-split, view-dependent cuts chosen in parallel, two-pass occlusion culling, a software rasterizer for tiny triangles, a visibility buffer and on-demand streaming.

Part II Lumen

How to light a fully dynamic world with multi-bounce indirect light at 30–60 fps: a surface cache of mesh cards, signed distance fields for cheap rays, screen-space probes for the final gather, a world-space radiance cache, and hybrid hardware ray tracing.

Sources. Brian Karis, Rune Stubbe & Graham Wihlidal, *A Deep Dive into Nanite Virtualized Geometry* (SIGGRAPH 2021 Advances); Daniel Wright, Krzysztof Narkowicz & Patrick Kelly, *Lumen: Real-time Global Illumination in Unreal Engine 5* (SIGGRAPH 2022 Advances); Epic Games UE5 documentation and release notes through 5.7. Numbers quoted (128 triangles per cluster, 8×8 probe octahedra, etc.) are the published defaults and are tunable.

Contents

1	Nanite: the problem	3
2	Building the cluster DAG (offline)	3
2.1	Clusters	3
2.2	Group, simplify, split	3
2.3	Why the result is a DAG, not a tree	4
3	Choosing the cut: view-dependent LOD	5
3.1	Error metric	5
3.2	The parallel selection rule	5
3.3	Hierarchical culling	5
4	Two-pass occlusion culling	6
5	Rasterization: software for tiny triangles	6
5.1	Why hardware is the wrong tool at one triangle per pixel	6
5.2	The split	6
5.3	Visibility buffer, not GBuffer	7
5.4	Deferred materials	7
6	Streaming, compression and the long tail	7
6.1	Virtualized geometry	7
6.2	Compression	7
6.3	Limits and later extensions	8
7	Lumen: the problem	9
8	Representing the scene for rays	9
8.1	Mesh distance fields	9
8.2	Global distance field	10
8.3	The surface cache	10
8.4	Radiosity in the cache: infinite bounces for free	10
9	The final gather: screen probes	11
9.1	Why probes	11
9.2	What a probe holds	11

9.3	Importance sampling the probe rays	11
9.4	Filtering and integration	12
10	The tracing ladder	12
10.1	Hardware ray tracing mode	13
11	Reflections, translucency and the rest	13
12	How the two fit together	14

The one-paragraph version

Both systems attack the same enemy: cost that scales with *content* rather than with *pixels*. Nanite makes geometry cost scale with screen resolution by preprocessing every mesh into a hierarchy of tiny clusters and, each frame, picking exactly the clusters whose simplification error is below one pixel. Lumen makes lighting cost scale with resolution by refusing to trace rays per pixel: it traces from a sparse set of screen probes, into a cheap proxy of the world (distance fields and a cached, pre-lit “surface” of each mesh), and reuses everything it can across space and time. Nanite supplies the dense geometry; Lumen is what makes it look lit.

1 Nanite: the problem

A traditional engine draws a mesh with a hardware draw call, picking one of a handful of artist-authored levels of detail (LODs) for the whole object. That design has three failure modes once source assets reach film density (millions of triangles per rock, thousands of rocks):

1. **Whole-object LOD is too coarse.** A cliff that is 2 m from the camera at its base and 400 m away at its top needs very different detail at each end.
2. **Small triangles are wasteful in hardware.** GPUs rasterize in 2×2 pixel quads; a triangle covering one pixel still pays for four, and the fixed-function setup rate becomes the bottleneck long before pixel shading does.
3. **Memory.** Film-quality assets do not fit in VRAM all at once.

Key idea

Make the number of triangles drawn scale with the number of *pixels*, not the number of triangles in the source art. The target is “one triangle per pixel”: any more is invisible, any fewer is visibly faceted. Everything in Nanite is machinery for choosing, culling, rasterizing and streaming geometry at that density.

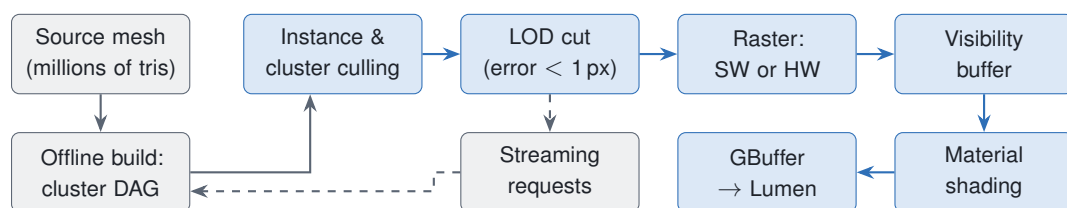


Figure 1: Nanite at a glance. Grey: offline or asynchronous; blue: every frame on the GPU. Culling and LOD selection are fused into one hierarchical traversal in practice.

2 Building the cluster DAG (offline)

2.1 Clusters

The source mesh is partitioned into **clusters** of up to 128 triangles each, using a graph partitioner (METIS) on the triangle adjacency graph so that clusters are spatially compact and share few boundary edges. A cluster is Nanite’s atom: it is culled, LOD-selected, streamed and rasterized as a unit. 128 triangles is small enough for fine-grained culling and large enough to amortize per-cluster work and to map well onto GPU workgroups.

2.2 Group, simplify, split

The naive way to get LODs per cluster would be to simplify each cluster independently. That fails immediately: two neighboring clusters at different LODs no longer share an edge, so cracks open between them. Locking cluster boundaries during simplification avoids cracks but then those boundary edges can *never* be simplified, and after a few levels the mesh is all boundary.

Nanite’s answer is the core trick of the whole system:

1. **Group** neighboring clusters (typically 8–32, roughly 4 is the conceptual minimum) into a cluster group, again with a graph partitioner that minimizes shared edges.
2. **Merge** each group's triangles and **simplify** the merged patch to half its triangle count with quadric error metrics, *locking only the group's outer boundary*. Interior edges between the old clusters vanish.
3. **Split** the simplified patch back into new 128-triangle clusters.
4. Repeat on the new clusters. Because the next grouping is computed fresh, the boundaries that were locked at this level land in the *interior* of groups at the next level, and get simplified there.

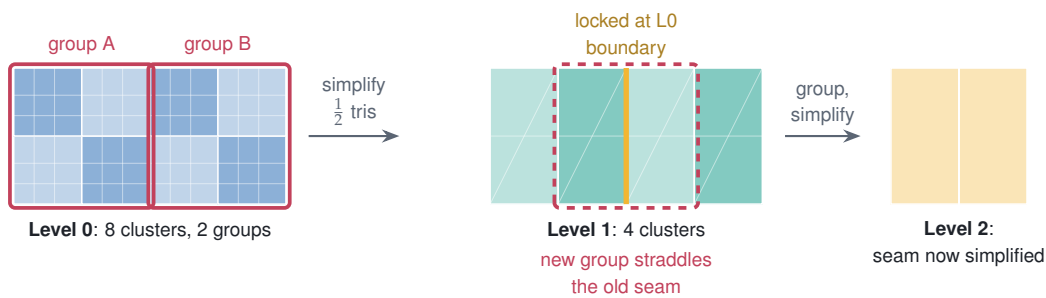


Figure 2: Group-simplify-split. Locking a group's boundary prevents cracks at this level; regrouping at the next level puts that boundary inside a group, so no edge stays locked forever. Each level halves the triangle count.

2.3 Why the result is a DAG, not a tree

A level- $(k+1)$ cluster is made from the merged triangles of a whole *group* of level- k clusters, so it has several “children”; and since the simplified group is re-split into several clusters, each level- k cluster contributes to several parents. The hierarchy is therefore a directed acyclic graph. The important structural fact is that **a group's clusters and the clusters produced from that group cover exactly the same surface with exactly the same boundary**. Swapping one set for the other is always watertight. LOD is decided per group, which is what lets neighboring regions sit at different detail without cracks.

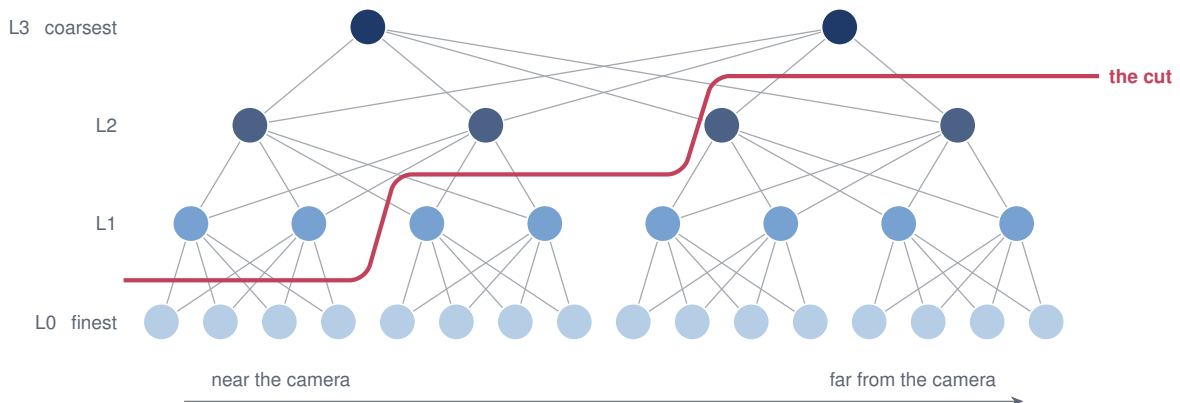


Figure 3: The cluster DAG (simplified: real groups overlap irregularly). A frame renders a *cut* through the DAG, fine where the surface is close, coarse where it is far. Every cut is crack-free because cuts only switch whole groups.

3 Choosing the cut: view-dependent LOD

3.1 Error metric

Every simplification step records a geometric error ϵ in object space (from the quadric error of the simplification). For a cluster c , Nanite stores its own error $\epsilon(c)$ and a bounding sphere, and also the error and bounds of the *group it was simplified into*, i.e. its parents, $\epsilon(\text{parent}(c))$. At runtime the error is projected to the screen. For a sphere of center $\mathbf{p}$, distance $d = \|\mathbf{p} - \mathbf{e}\|$ from the eye, a vertical field of view θ and a viewport H pixels tall,

$$\epsilon_{\text{px}}(c) = \frac{\epsilon(c) H}{2 d \tan(\theta/2)}. \quad (1)$$

(The real implementation uses the closest point of the bounding sphere, and accounts for instance scale.)

3.2 The parallel selection rule

Walking a DAG top-down on a GPU is awkward. Nanite instead makes the decision for every cluster *independently*:

$$\boxed{\text{draw } c \iff \epsilon_{\text{px}}(c) \leq \tau \text{ and } \epsilon_{\text{px}}(\text{parent}(c)) > \tau} \quad (\tau \approx 1 \text{ pixel}). \quad (2)$$

“I am precise enough, and my parent is not.” This yields exactly one consistent cut *provided* that the projected error is monotonic along the DAG: a parent must always appear at least as erroneous as its children from any viewpoint. Nanite enforces this at build time by making each group’s stored error the max of its children’s errors and its bounding sphere enclose its children’s spheres. Then for any view there is precisely one level on every root-to-leaf path where the error crosses τ , all clusters of a group make the same decision about their shared parent, and the cut is watertight.

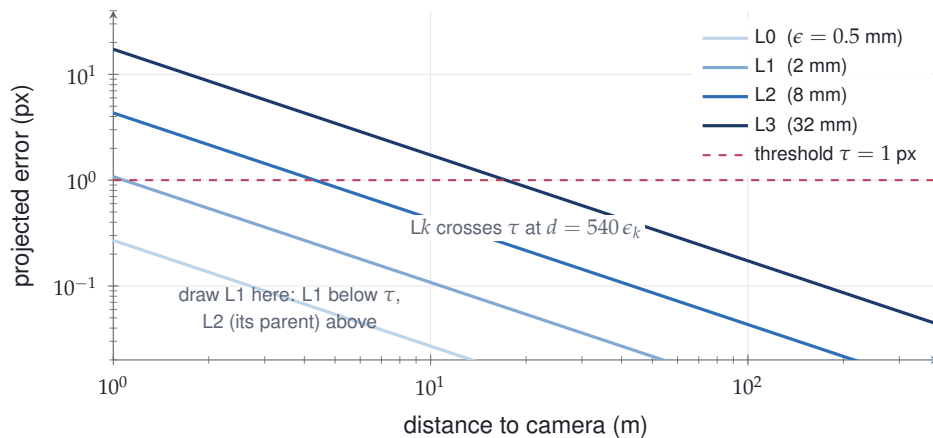


Figure 4: Projected error falls as $1/d$. For each level the band where it is the *coarsest* level still under one pixel is where it is drawn ($H = 1080$, 90° FOV). Because each level halves triangle count and the switch distance doubles with error, the triangle count per pixel stays roughly constant.

3.3 Hierarchical culling

Testing every cluster in the scene each frame would still be millions of tests. The cluster groups are placed in a bounding-volume hierarchy (a wide BVH, 4 to 8 children per node). Traversal discards

a whole subtree if its bounds are outside the frustum, occluded, or if *every* cluster beneath it would already be too fine (the parent error at that node is below τ , so nothing beneath it can be the cut). Traversal runs as a single persistent compute dispatch with a global work queue, so it does not need one dispatch per tree level.

4 Two-pass occlusion culling

Nanite culls against a hierarchical Z-buffer (HZB): a mip pyramid of the depth buffer where each texel stores the *farthest* depth beneath it. A cluster's screen rectangle is tested against the mip level where it covers about 2×2 texels; if its nearest depth is behind the stored farthest depth, it is hidden.

The difficulty is that the current frame's depth does not exist until you have drawn the frame. Nanite resolves the chicken-and-egg with two passes:

Algorithm 1: Two-pass occlusion culling (per frame)

Input: HZB_{prev} from last frame; set V_{prev} of instances/clusters visible last frame

```

1 Pass 1 ;                                ▷ guess: what was visible stays visible
2 foreach node in  $V_{prev}$  do
3   | if in frustum and not occluded by HZBprev (reprojected) then
4   |   | rasterize
5 Build HZBcur from the depth just rendered;

6 Pass 2 ;                                ▷ catch everything the guess got wrong
7 foreach node not drawn in pass 1 do
8   | if in frustum and not occluded by HZBcur then
9   |   | rasterize; mark visible
10 Rebuild HZBcur; store  $V_{cur}$  for next frame;
```

The first pass gets most of the frame right cheaply because consecutive frames are coherent. The second pass guarantees correctness: anything newly disoccluded is tested against a depth buffer that is *conservative for this frame* (it contains only things that really are visible now), so nothing visible is ever culled. The cost is one extra HZB build and a second, usually tiny, traversal.

5 Rasterization: software for tiny triangles

5.1 Why hardware is the wrong tool at one triangle per pixel

Fixed-function rasterizers are built for triangles covering tens to hundreds of pixels: setup cost is paid once per triangle, then many pixels are filled in 2×2 quads. At one pixel per triangle, setup throughput and quad overshading dominate. Nanite measured its compute-shader rasterizer at roughly three times faster than the hardware path for micro-triangles.

5.2 The split

Each visible cluster is binned by the screen size of its triangles. Clusters of small triangles go to a **software rasterizer** in a compute shader (one thread per triangle, scanline or edge-function iteration over a tiny bounding box). Clusters with large triangles go to the hardware rasterizer,

where it is still faster.

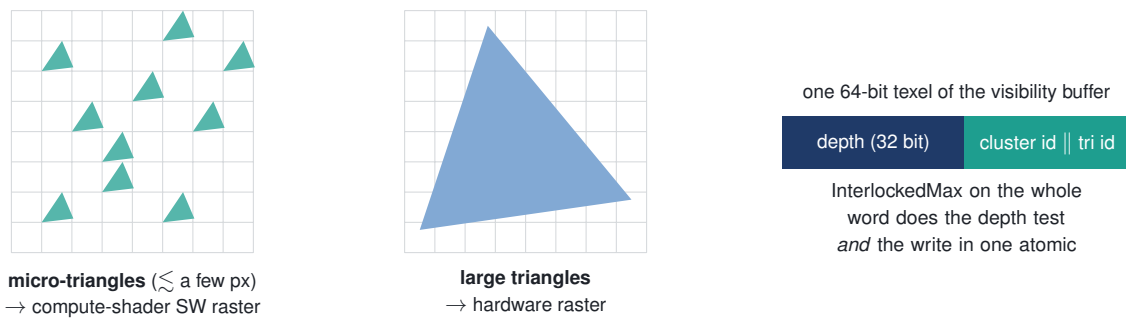


Figure 5: Left: the software path wins where hardware would pay full setup and a 2×2 quad for a one-pixel triangle. Right: the visibility-buffer encoding that makes a software depth test possible without ROPs.

5.3 Visibility buffer, not GBuffer

Neither rasterizer shades anything. They write a **visibility buffer**: per pixel, a 64-bit word with depth in the high bits and (visible-cluster index, triangle index) in the low bits. Because depth occupies the most significant bits, a single 64-bit atomic max (with reversed-Z, nearer = larger) both depth-tests and writes the payload, which is how a compute shader can rasterize without fixed-function ROPs and without races.

5.4 Deferred materials

Shading happens afterwards, once per pixel. From the triangle id, a material pass fetches the three vertices, recomputes barycentrics and analytic derivatives for that pixel, interpolates attributes and runs the material shader, writing a standard GBuffer that the rest of the renderer (and Lumen) consumes. The original UE5 implementation drew one full-screen pass per material, using a material-ID depth test so each pass only touched its own pixels; later versions bin pixels by material and shade in compute. Two consequences: overdraw costs only a 64-bit atomic, never a material evaluation; and material count has a per-frame overhead that grows with the number of distinct materials visible.

6 Streaming, compression and the long tail

6.1 Virtualized geometry

Like virtual texturing, Nanite keeps only what the current views need. Cluster groups are packed into fixed-size pages (128 KB). A small set of *root pages* (the coarsest levels) is always resident so any mesh can always draw something. During culling, whenever the cut wants a cluster whose data is not resident, the GPU writes a request; the CPU streams pages in priority order and the cut refines over the next few frames. The DAG structure guarantees that a coarser, resident ancestor can always stand in.

6.2 Compression

Positions are quantized to a per-mesh grid and stored with per-cluster bit widths relative to the cluster's bounds; normals use octahedral encoding; indices are stored locally within the cluster's

vertex set. The in-memory format is decodable directly on the GPU, and the on-disk format adds a byte-oriented layout that general-purpose compressors (Oodle) shrink further. Published figures put a Nanite mesh at around 14 bytes per input triangle on disk, smaller than a typical non-Nanite mesh with its LOD chain.

6.3 Limits and later extensions

- **Aggregate geometry.** Nanite simplifies surfaces. Grass, leaves and hair are many disconnected thin pieces; simplifying them erodes volume and they cannot be culled by self-occlusion. Foliage support arrived progressively (masked materials, programmable raster, and in 5.7 an experimental foliage path that adds instanced assemblies, skinned deformation and a voxel-like representation for distant canopies).
- **Deformation.** Static meshes first; skeletal meshes, World Position Offset and displacement-based tessellation were added in 5.x releases.
- **Faceting at the cut.** Because LOD switches per group, topology can change slightly as you move; at $\tau \approx 1$ px this is generally invisible, but hard-edged detail like engraved text can shimmer.
- **Shadows.** Nanite pairs with Virtual Shadow Maps ($16k^2$ virtual shadow maps rendered through the same Nanite pipeline), since conventional shadow maps cannot resolve micro-geometry.

7 Lumen: the problem

Global illumination means solving the rendering equation, not just the direct term:

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} f_r(\mathbf{x}, \omega_i, \omega_o) L_i(\mathbf{x}, \omega_i) (\mathbf{n} \cdot \omega_i) d\omega_i, \quad (3)$$

$$L_i(\mathbf{x}, \omega_i) = L_o(\text{trace}(\mathbf{x}, \omega_i), -\omega_i).$$

The recursion inside L_i is what makes light bounce. Offline renderers estimate the integral by Monte Carlo with hundreds of rays per pixel. A game at 1440p and 60 fps can afford perhaps *one-half* of a ray per pixel, against a world that is fully dynamic (no baked lightmaps) and made of Nanite geometry far too dense to put in a ray-tracing structure at full detail.

Key idea

Never trace the real scene at full detail, and never trace once per pixel. Trace from a sparse set of probes; trace against cheap proxies (signed distance fields) whose hits can be looked up in a pre-lit *surface cache*; let the cache feed back into itself to get infinite bounces over time; and spend expensive rays only where they matter, ordered from cheapest-and-most-accurate to cheapest-and-coarsest.

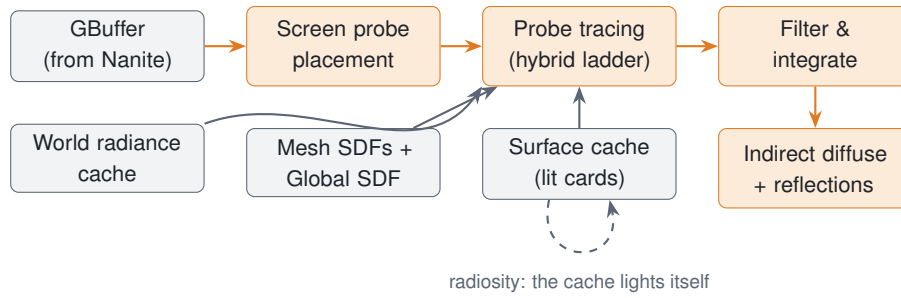


Figure 6: Lumen at a glance. Orange: per-frame screen-space work. Grey: world-space data structures, updated incrementally on a budget.

8 Representing the scene for rays

8.1 Mesh distance fields

Each mesh gets a precomputed **signed distance field** (SDF): a sparse 3D grid where each texel stores the distance to the nearest surface (negative inside). Ray marching an SDF is *sphere tracing*: at the current point, the stored distance is a radius in which nothing can be hit, so the ray can safely jump that far.

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \text{SDF}(\mathbf{p}_k) \omega, \quad \text{hit when } \text{SDF}(\mathbf{p}_k) < \varepsilon. \quad (4)$$

In open space the steps are huge; near surfaces they shrink. SDFs also give a nearly free soft-shadow and cone estimate, since the closest approach of a ray to geometry, $\min_k \text{SDF}(\mathbf{p}_k) / t_k$, approximates how much of a cone around the ray is blocked.

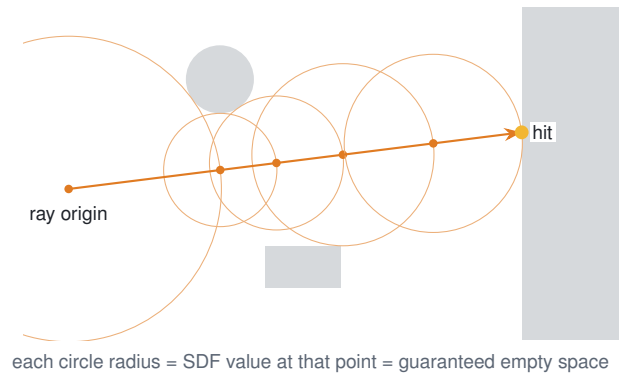


Figure 7: Sphere tracing a distance field: big steps through empty space, convergence near the surface. No triangles are touched.

8.2 Global distance field

Tracing dozens of overlapping mesh SDFs per ray gets expensive with distance. Lumen also merges all mesh SDFs into a **global distance field**: a set of camera-centered clipmaps (nested cubes of coarser resolution farther out), updated incrementally when objects move. It is cheaper and covers long range, but is low resolution and loses thin features. Lumen therefore uses precise mesh SDFs for the first couple of meters of a ray and switches to the global field beyond.

8.3 The surface cache

A distance-field hit tells you *where* the ray stopped, not *what color the light is there*. Evaluating full material shading and direct lighting at arbitrary hit points would be far too slow. Lumen’s answer is the **surface cache**:

- Each mesh is covered by a handful of **cards**: oriented rectangles placed around it (conceptually like an orthographic camera looking at the mesh from up to six sides; complex meshes get more cards from a build-time placement pass).
- Each card is rasterized (with Nanite, so it is cheap) into an atlas storing albedo, normal, emissive, opacity and depth. Card resolution follows distance to the camera and is updated on a per-frame budget.
- Lighting is then computed *in card space*: direct lighting with shadows, plus indirect lighting gathered by tracing short rays from the card texels themselves.

8.4 Radiosity in the cache: infinite bounces for free

When the surface cache computes indirect lighting for its own texels, it traces rays that themselves hit the surface cache and read *last frame’s* lit result. One bounce per frame accumulates into multi-bounce light within a few frames, a fixed-point iteration of the rendering equation over time:

$$L_{\text{cache}}^{(n+1)} = L_{\text{direct}} + \mathcal{T}[L_{\text{cache}}^{(n)}], \quad (5)$$

where $\mathcal{T}$ is the one-bounce transport operator. Since $\mathcal{T}$ has norm below one for physical albedos, the iteration converges to the full multi-bounce solution. Updates are amortized: only a budgeted subset of cards is relit each frame, prioritized by distance and recent change, which is why Lumen’s indirect light visibly “settles” a beat after a light switches on.

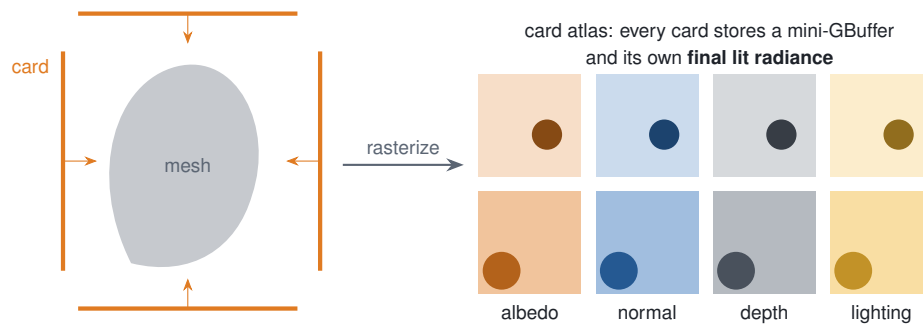


Figure 8: Cards capture each mesh from a few directions into an atlas. A ray that hits a mesh distance field projects the hit into that mesh's cards and reads the already-computed lighting: a texture fetch instead of a material evaluation.

9 The final gather: screen probes

9.1 Why probes

Tracing even one ray per pixel and denoising leaves too much noise for diffuse GI. Lumen exploits the fact that indirect diffuse light is low-frequency: it places **screen probes** sparsely (by default one per 16×16 pixel tile, plus extra probes where depth or normal change sharply), traces a full hemisphere from each, and interpolates between them per pixel.

9.2 What a probe holds

Each probe stores incoming radiance over the hemisphere in an **octahedral map** (the sphere unfolded onto a square), typically 8×8 directions. With 64 rays per probe at one probe per 256 pixels, that is $64/256 = 0.25$ rays per pixel, yet each pixel effectively integrates over many more directions because it interpolates neighboring probes, and probes are reused temporally.

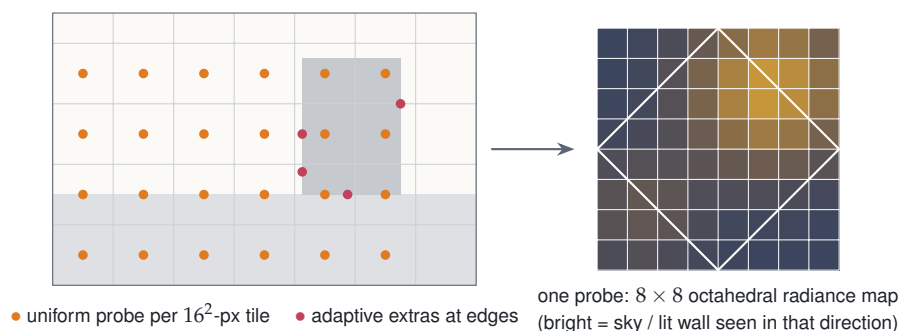


Figure 9: Screen probes sample incoming light sparsely; each stores a tiny directional image of the world around it.

9.3 Importance sampling the probe rays

Where should a probe's 64 rays go? Uniform directions waste rays on dark parts of the hemisphere. Lumen builds a sampling PDF from two terms it already knows: the BRDF and normal of the pixels that will use the probe, and the probe's *own radiance from last frame* (reprojected). Directions that were bright or that the BRDF weights heavily get more rays; the Monte Carlo estimate stays

unbiased by dividing by that PDF,

$$\hat{E}(\mathbf{x}) = \frac{1}{N} \sum_{k=1}^N \frac{L_i(\mathbf{x}, \omega_k) (\mathbf{n} \cdot \omega_k)}{p(\omega_k)}, \quad p(\omega) \propto \underbrace{\cos \theta}_{\text{BRDF}} \times \underbrace{\tilde{L}_{\text{prev}}(\omega)}_{\text{last frame}}. \quad (6)$$

This is the same product-importance-sampling idea used offline; doing it per probe rather than per pixel keeps the cost affordable.

9.4 Filtering and integration

Probe radiance is filtered spatially across neighboring probes (rejecting neighbors whose depth/normal disagree, and whose rays would have been occluded), converted to a compact form for interpolation (low-order spherical harmonics for diffuse), and then each pixel interpolates its nearby probes with plane-distance weights. A final temporal filter accumulates over frames.

10 The tracing ladder

Every probe ray is traced through a sequence of increasingly coarse but increasingly long-range methods. Each stage picks up where the previous one gave up.

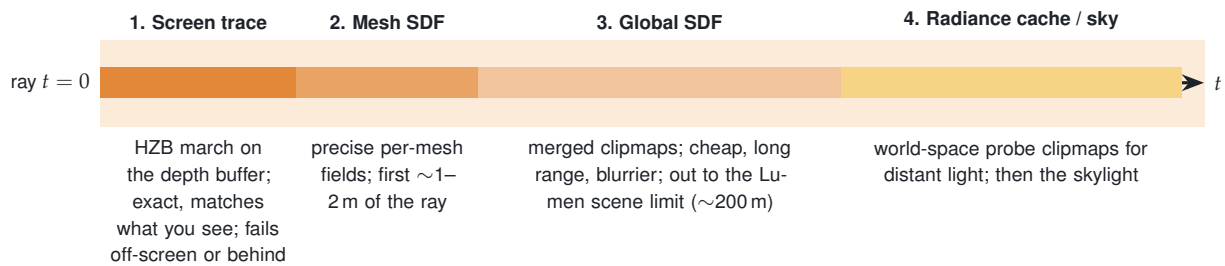


Figure 10: Lumen’s software tracing ladder for each probe ray. Hits from stages 2–3 read lighting from the surface cache; misses fall through to the next stage.

1. Screen traces. March the ray through the HZB of the current depth buffer. If it hits something on screen, read last frame’s final lit color there. This is the most accurate source because it matches the actual rendered geometry (Nanite detail included) and catches small-scale contact bounce that coarse proxies miss.

2–3. Distance-field traces. If the screen trace leaves the screen or passes behind a surface, continue from where it stopped, through mesh SDFs and then the global SDF, and shade hits with a surface-cache lookup.

4. Radiance cache. Long rays are both the most expensive and the least important individually. Lumen keeps a second, *world-space* probe set: sparse probes on camera-centered clipmaps, traced at higher angular resolution but updated slowly and reused by every screen probe. A screen-probe ray that travels past a set distance stops and samples the world radiance cache instead of tracing on. This is what makes small, bright, distant features (a lit window across a dark room) stable rather than noisy.

10.1 Hardware ray tracing mode

On RT-capable GPUs, stages 2–3 can be replaced by ray tracing real triangles through the hardware BVH. Nanite meshes enter the BVH via a simplified fallback mesh (the full micro-polygon detail would be too large), while screen traces still recover full detail near the surface. Hits can either read the surface cache (fast) or evaluate full material and lighting at the hit (“hit lighting”, slower, needed for mirror-like reflections). Hardware mode handles thin geometry, skinned meshes and accurate occlusion far better; software mode runs anywhere and scales to more instances. Recent releases (5.6, 5.7) have been shifting the default weight toward the hardware path as its console performance improved.

11 Reflections, translucency and the rest

Reflections. Rough reflections are just indirect specular with a wide lobe, so they reuse the screen-probe radiance. Smoother surfaces (roughness below a threshold) get dedicated reflection rays traced per pixel at reduced resolution through the same ladder, spatially and temporally denoised. Mirror reflections in hardware mode use hit lighting so that reflected objects are fully shaded rather than looked up in the coarse surface cache.

Translucency and fog. A low-resolution froxel volume (frustum-aligned voxels) is lit by tracing from voxel centers into the same structures; translucent surfaces and volumetric fog sample it.

Where it breaks. Thin walls below the SDF resolution leak light; meshes whose shape cards cannot cover (deep concavities) show missing or dark indirect light; fast lighting changes lag because the surface cache and temporal filters converge over several frames; and very large emissive-only scenes are noisy because emissive surfaces are found only by chance rays, not sampled directly. (MegaLights, introduced in 5.5 and extended in 5.7, addresses direct lighting from many lights, a separate problem.)

12 How the two fit together

	Nanite	Lumen
Problem	Geometry cost scales with content	Lighting cost scales with rays per pixel
Core trick	Cluster DAG + one-pixel error cut	Sparse probes + proxy scene + caches
Hierarchy	Cluster DAG, BVH over groups	SDF clipmaps, probe clipmaps, mip HZB
Reuse over time	Last frame's visible set (pass 1)	Surface cache, radiance cache, temporal filters, importance PDF
Reuse over space	One cluster serves many pixels	One probe serves ~256 pixels
GPU style	Persistent compute, 64-bit atomics	Compute probe tracing, async updates
Bottleneck it removes	Triangle setup, overdraw, VRAM	Ray count, material evaluation at hits
Typical artifact	Rare LOD topology shimmer	Light leaks, lag after sudden changes

The dependency runs both ways. Lumen needs Nanite: cards are captured by rasterizing meshes, and doing that for thousands of cards a frame is only affordable because Nanite draws them cheaply. Nanite needs Lumen (or something like it): film-density geometry lit with flat ambient light looks like a plastic model; the micro-occlusion and bounce light that Lumen supplies is what sells the detail.

The shared design principle

Both are *output-sensitive* algorithms. They do not ask “how much is in the world?” but “how much can the screen show?” and then use hierarchy (to make that question cheap to answer), conservative bounds (so cheap answers are never wrong in the dangerous direction), and temporal coherence (so most of this frame's answer is last frame's answer). It is the same pattern as virtual memory and virtual texturing, applied to triangles and to light.

Further reading. Karis, Stubbe, Wihlidal, *A Deep Dive into Nanite Virtualized Geometry*, SIGGRAPH 2021 Advances in Real-Time Rendering. Wright, Narkowicz, Kelly, *Lumen: Real-time Global Illumination in Unreal Engine 5*, SIGGRAPH 2022 Advances. Burns & Hunt, *The Visibility Buffer*, JCGT 2013. Hart, *Sphere Tracing*, The Visual Computer 1996. Cigolle et al., *A Survey of Efficient Representations for Independent Unit Vectors* (octahedral encoding), JCGT 2014. Epic Games, Unreal Engine 5.7 release notes.

図解

Nanite & Lumen

仮想化マイクロポリゴン・ジオメトリと
完全動的グローバルイルミネーション
Unreal Engine 5 を支える二つのアルゴリズム

第I部 Nanite

元データで数十億に達する三角形を、ほぼ1ピクセルに1三角形の密度で描く方法。分割・簡略化・再分割の反復で作るクラスタ DAG、並列に選ばれる視点依存のカット、2パスのオクルージョンカリング、微小三角形用のソフトウェアラスタライザ、ビジビリティバッファ、オンデマンドのストリーミング。

第II部 Lumen

完全に動的な世界を、多重反射する間接光込みで30~60 fpsで照らす方法。メッシュのカードによるサーフェスキャッシュ、安価なレイのための符号付き距離場、最終ギャザーのためのスクリーンプロブ、ワールド空間のラディアンスキャッシュ、そしてハードウェアレイトレーシングとの併用。

出典 Brian Karis, Rune Stubbe & Graham Wihlidal, A Deep Dive into Nanite Virtualized Geometry (SIGGRAPH 2021 Advances)、Daniel Wright, Krzysztof Narkowicz & Patrick Kelly, Lumen: Real-time Global Illumination in Unreal Engine 5 (SIGGRAPH 2022 Advances)、Epic Games の UE5 ドキュメントと 5.7 までのリリースノート。本文中の数値（クラスタあたり128三角形、プロブあたり8×8の八面体マップなど）は公表された既定値であり、調整可能である。

目次

1 Nanite : 問題設定	4
2 クラスタ DAG の構築 (オフライン)	4
2.1 クラスタ	4
2.2 グループ化、簡略化、再分割	5
2.3 結果が木ではなく DAG になる理由	5
3 カットの選択 : 視点依存の LOD	6
3.1 誤差の尺度	6
3.2 並列な選択規則	6
3.3 階層的カリング	7
4 2パスのオクルージョンカリング	7
5 ラスタライズ : 微小三角形にはソフトウェアを	8
5.1 1ピクセル1三角形ではハードウェアが不向きな理由	8
5.2 振り分け	8
5.3 GBuffer ではなくビジビリティバッファ	8
5.4 遅延マテリアル	9
6 ストリーミング、圧縮、そして残された課題	9
6.1 仮想化ジオメトリ	9
6.2 圧縮	10
6.3 限界とその後の拡張	10
7 Lumen : 問題設定	11
8 レイのためのシーン表現	11
8.1 メッシュ距離場	11
8.2 グローバル距離場	12
8.3 サーフェスキャッシュ	12
8.4 キャッシュ内のラジオシティ : 無限反射をただで	13
9 最終ギャザー : スクリーンプローブ	13

9.1 なぜプローブか	13
9.2 プローブが保持するもの	13
9.3 プローブレイの重点サンプリング	14
9.4 フィルタリングと積分	14
10 トレースのはしご	14
10.1 ハードウェアレイトレーシング・モード	15
11 反射、半透明、その他	15
12 二つはどう噛み合うか	17

一段落でまとめると

二つのシステムが相手にしているのは同じ敵である。すなわち、ピクセル数ではなくコンテンツ量に比例して増えるコストだ。Nanite は、あらゆるメッシュを微小なクラスタの階層へ前処理し、毎フレーム「簡略化誤差が1ピクセル未満となるクラスタ」だけを正確に選ぶことで、ジオメトリのコストを画面解像度に比例させる。Lumen は、ピクセルごとにレイを飛ばすことを拒むことで、ライティングのコストを解像度に比例させる。疎に配置したスクリーンプローブから、世界の安価な代理表現（距離場と、メッシュごとに事前にライティング済みの「サーフェス」のキャッシュ）に向けてレイを飛ばし、空間方向にも時間方向にも再利用できるものはすべて再利用する。Nanite が密なジオメトリを供給し、Lumen がそれを照らされたものに見せる。

1 Nanite : 問題設定

従来のエンジンは、メッシュをハードウェアのドローコールで描き、アーティストが用意した数段階の詳細度 (LOD) のうち一つをオブジェクト全体に適用する。素材が映画並みの密度 (岩一つに数百万三角形、その岩が数千個) に達すると、この設計は三つの点で破綻する。

1. **オブジェクト単位の LOD は粗すぎる。** 根元がカメラから 2 m、頂上が 400 m 離れた崖は、両端でまったく異なる詳細度を必要とする。
2. **小さな三角形はハードウェアにとって無駄が多い。** GPU は 2×2 ピクセルのクアッド単位でラスタライズするため、1 ピクセルしか覆わない三角形でも 4 ピクセル分の費用を払う。ピクセルシェーディングよりはるか手前で、固定機能のセットアップ速度がボトルネックになる。
3. **メモリ。** 映画品質の素材は、一度にすべて VRAM に載らない。

要点

描画する三角形の数を、元素材の三角形数ではなくピクセル数に比例させる。目標は「1 ピクセルに 1 三角形」である。それより多くても見えず、少なければ角張って見える。Nanite のすべては、この密度でジオメトリを選び、カリングし、ラスタライズし、ストリーミングするための仕組みである。

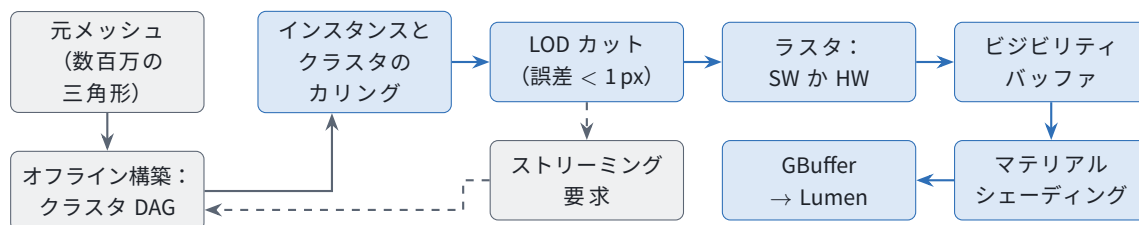


図 1 Nanite の全体像。灰色はオフラインまたは非同期の処理、青は毎フレーム GPU 上で行われる処理。実際にはカリングと LOD 選択は一つの階層走査に統合されている。

2 クラスタ DAG の構築 (オフライン)

2.1 クラスタ

元のメッシュは、三角形の隣接グラフにグラフ分割器 (METIS) をかけることで、最大 128 三角形からなる **クラスタ** に分割される。クラスタは空間的にまとまり、境界の辺をなるべく共有しないように作られる。クラスタは Nanite の原子であり、カリング、LOD 選択、ストリーミング、ラスタライズはすべてクラスタ単位で行われる。128 三角形という大きさは、細かいカリングができる程度に小さく、クラスタごとの処理を償却し GPU のワークグループにうまく対応する程度に大きい。

2.2 グループ化、簡略化、再分割

各クラスタを独立に簡略化して LOD を作るという素朴な方法は、すぐに破綻する。隣り合う二つのクラスタが異なる LOD になると辺が一致しなくなり、間に亀裂が生じるからだ。簡略化の際にクラスタ境界を固定すれば亀裂は防げるが、今度はその境界の辺が永遠に簡略化されず、数段階もするとメッシュは境界だらけになる。

Nanite の答えは、システム全体の核となる次の手法である。

1. 隣接するクラスタ（典型的には8〜32個。概念上の最小は4個程度）を、共有辺を最小にするグラフ分割で**クラスタグループ**にまとめる。
2. 各グループの三角形を**結合**し、二次誤差計量（QEM）で三角形数が半分になるまで**簡略化**する。このとき固定するのはグループの外周だけであり、旧クラスタ間の内部の辺は消える。
3. 簡略化したパッチを、再び128三角形のクラスタに**再分割**する。
4. 新しいクラスタに対して同じ手順を繰り返す。次のグループ分けは改めて計算されるため、この段で固定された境界は次の段ではグループの内部に入り、そこで簡略化される。

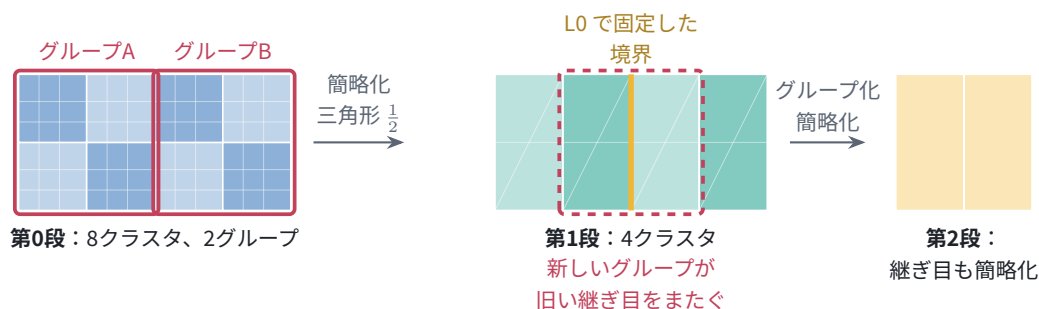


図2 グループ化・簡略化・再分割。グループの境界を固定することでその段の亀裂を防ぎ、次の段で組み直すことでその境界がグループの内部に入るため、固定され続ける辺は存在しない。各段で三角形数は半分になる。

2.3 結果が木ではなく DAG になる理由

第 $k+1$ 段のクラスタは第 k 段のクラスタグループ全体の三角形から作られるので、複数の「子」をもつ。また簡略化されたグループは複数のクラスタに再分割されるので、第 k 段の各クラスタは複数の親に寄与する。したがって階層は有向非巡回グラフ（DAG）となる。構造上もっとも重要な事実は、あるグループのクラスタ群と、そのグループから作られたクラスタ群は、まったく同じ境界をもつまったく同じ曲面を覆うということである。一方を他方に置き換えても、常に隙間は生じない。LOD はグループ単位で決まり、これによって隣接する領域が亀裂なしに異なる詳細度をとれる。

3 カットの選択：視点依存の LOD

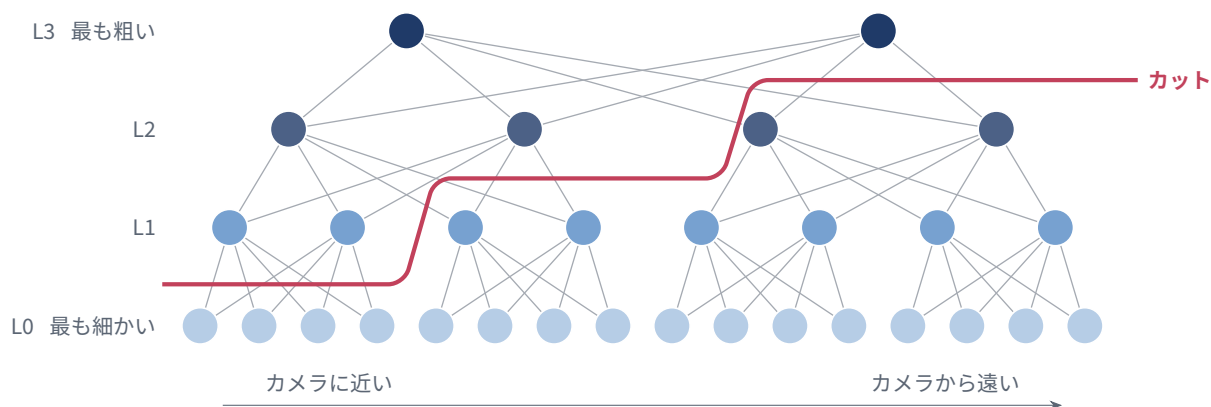


図3 クラスタ DAG (簡略図。実際のグループは不規則に重なる)。各フレームは DAG を通るカットを描画する。曲面が近いところは細かく、遠いところは粗い。カットは常にグループ単位でしか切り替わらないため、どのカットにも亀裂はない。

3.1 誤差の尺度

簡略化の各段は、オブジェクト空間での幾何誤差 ϵ (簡略化の二次誤差から得る) を記録する。各クラスタ c について、Nanite は自身の誤差 $\epsilon(c)$ とバウンディング球を保持し、さらに自身が簡略化されてできたグループ、つまり親の誤差 $\epsilon(\text{parent}(c))$ と境界も保持する。実行時には誤差を画面に投影する。中心 $\mathbf{p}$ 、視点からの距離 $d = \|\mathbf{p} - \mathbf{e}\|$ 、垂直視野角 θ 、高さ H ピクセルのビューポートに対して、

$$\epsilon_{\text{px}}(c) = \frac{\epsilon(c) H}{2 d \tan(\theta/2)}. \quad (1)$$

(実装では、バウンディング球のもっとも近い点を用い、インスタンスのスケールも考慮する。)

3.2 並列な選択規則

GPU 上で DAG を上から順に辿るのは扱いにくい。そこで Nanite は、すべてのクラスタについて独立に判定を下す。

$$c \text{ を描く} \iff \epsilon_{\text{px}}(c) \leq \tau \text{ かつ } \epsilon_{\text{px}}(\text{parent}(c)) > \tau \quad (\tau \approx 1 \text{ ピクセル}). \quad (2)$$

「自分は十分に精密で、親はそうではない」。これは、投影誤差が DAG に沿って単調であるならば、ちょうど一つの整合したカットを与える。つまり、どの視点から見ても親は子と同じかそれ以上に誤差が大きく見えなければならない。Nanite は構築時に、各グループに保存する誤差を子の誤差の最大値とし、そのバウンディング球が子の球を包むようにすることで、これを保証する。すると、どの視点についても根から葉へのすべての経路上で誤差が τ をまたぐ段がちょうど一つ存在し、あるグループのクラスタはすべて共通の親について同じ判定を下し、カットは隙間なく閉じる。

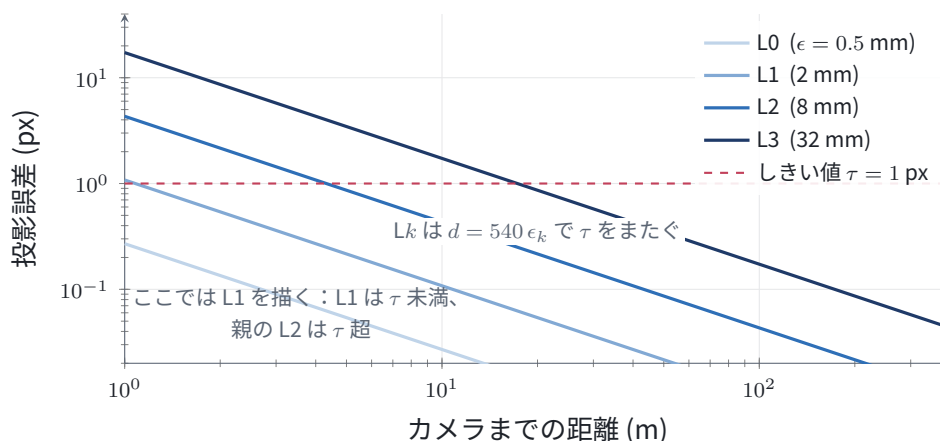


図 4 投影誤差は $1/d$ で減少する。各段は、1ピクセル未満に収まるもっとも粗い段となる距離帯で描かれる ($H = 1080$ 、視野角 90°)。段ごとに三角形数は半分になり、切り替え距離は誤差とともに倍になるので、ピクセルあたりの三角形数はほぼ一定に保たれる。

3.3 階層のカリング

毎フレーム、シーン内の全クラスタを判定するのでは、それでも数百万回の判定になる。クラスタグループはバウンディングボリューム階層（子を4～8個もつ幅の広い BVH）に格納される。走査では、境界が視錐台の外にある場合、遮蔽されている場合、またはその下のすべてのクラスタがすでに細かすぎる場合（そのノードでの親誤差が τ 未満なので、下にカットはありえない）に、部分木ごとと捨てる。走査はグローバルな作業キューをもつ単一の常駐型コンピュートディスパッチとして実行されるため、木の段ごとにディスパッチを発行する必要がない。

4 2パスのオクルージョンカリング

Nanite は階層 Z バッファ (HZB) に対してカリングを行う。HZB は深度バッファのミップピラミッドで、各テクセルはその下にあるもっとも遠い深度を保持する。クラスタの画面上の矩形は、それがおよそ 2×2 テクセルを覆うミップ段で判定され、もっとも近い深度が保存された最遠深度より奥にあれば隠れていると判断される。

難しいのは、現在のフレームの深度は、そのフレームを描き終えるまで存在しないという点である。Nanite は、この鶏と卵の問題を2パスで解決する。

アルゴリズム 1: 2パスのオクルージョンカリング（フレームごと）

入力: 前フレームの HZB_{prev} 、前フレームで可視だったインスタンス／クラスタの集合 V_{prev}

- 1 **パス1;** ▷ 推測：見えていたものは引き続き見える
- 2 各 V_{prev} のノード について
- 3 もし 視錐台内 かつ HZB_{prev} （再投影）で遮蔽されていない ならば
- 4 └ ラスタライズ
- 5 いま描いた深度から HZB_{cur} を構築;
- 6 **パス2;** ▷ 推測が外れたものをすべて拾う
- 7 各 パス1で描かれなかったノード について
- 8 もし 視錐台内 かつ HZB_{cur} で遮蔽されていない ならば
- 9 └ ラスタライズし、可視として記録
- 10 HZB_{cur} を再構築し、次フレームのために V_{cur} を保存;

連続するフレームには一貫性があるので、パス1だけで大半を安く正しく描ける。パス2が正しさを保証する。新たに見えるようになったものは、このフレームにとって保守的な深度バッファ（いま本当に見えているものだけを含む）に対して判定されるので、見えるものが誤って捨てられることはない。代償は、HZB の構築が1回増えることと、通常はごく小さい2回目の走査である。

5 ラスタライズ：微小三角形にはソフトウェアを

5.1 1ピクセル1三角形ではハードウェアが不向きな理由

固定機能のラスタライザは、数十から数百ピクセルを覆う三角形のために作られている。セットアップの費用は三角形ごとに一度払い、その後 2×2 のクアッドで多くのピクセルを埋める。三角形が1ピクセル程度になると、セットアップのスループットとクアッドの過剰シェーディングが支配的になる。Nanite のコンピュートシェーダ製ラスタライザは、微小三角形についてハードウェア経路のおよそ3倍速いと計測されている。

5.2 振り分け

可視クラスタは、その三角形の画面上の大きさで振り分けられる。小さな三角形のクラスタはコンピュートシェーダのソフトウェアラスタライザへ送られ（三角形ごとに1スレッドで、小さなバウンディングボックス内を走査線またはエッジ関数で反復する）、大きな三角形のクラスタは、まだそちらが速いハードウェアラスタライザへ送られる。

5.3 GBuffer ではなくビジビリティバッファ

どちらのラスタライザも何もシェーディングしない。書き込むのはビジビリティバッファであり、ピクセルごとに64ビットの語の上位ビットに深度を、下位ビットに（可視クラスタ番号、三角形番号）を入れる。深度が最上位ビットを占めるので、64ビットのアトミック最大値演算

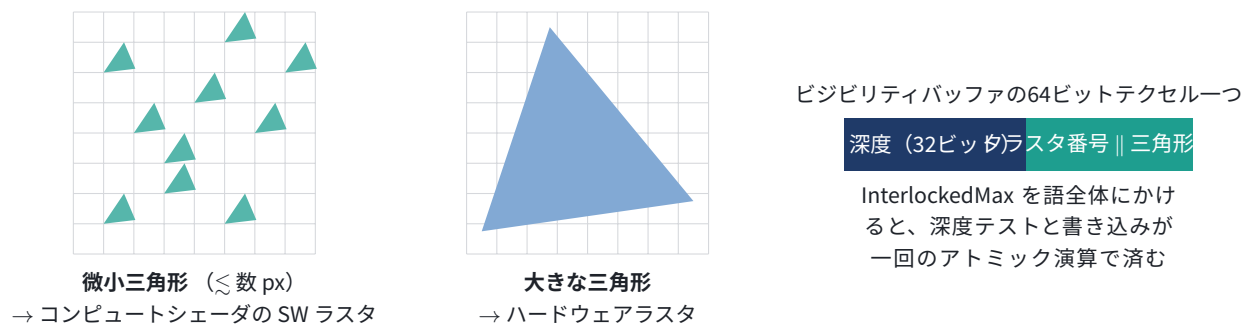


図5 左：ハードウェアなら1ピクセルの三角形にもセットアップ全体と 2×2 クアッドを払うところで、ソフトウェア経路が勝つ。右：ROP なしでソフトウェアの深度テストを可能にするビジビリティバッファの符号化。

一回（リバース Z では近いほど大きい）で深度テストと書き込みが同時に済む。これが、固定機能の ROP なしに、競合も起こさずにコンピュートシェーダでラスタライズできる理由である。

5.4 遅延マテリアル

シェーディングはその後、ピクセルごとに一度だけ行われる。三角形番号からマテリアルパスが三頂点を取り出し、そのピクセルについて重心座標と解析的な微分を再計算し、属性を補間してマテリアルシェーダを実行し、標準的な GBuffer に書き込む。レンダラの残り（と Lumen）はそれを使う。初期の UE5 実装ではマテリアルごとに全画面パスを1回描き、マテリアル ID の深度テストで各パスが自分のピクセルだけに触れるようにしていた。後のバージョンでは、ピクセルをマテリアル別にビン分けしてコンピュートでシェーディングする。帰結は二つある。オーバードローの費用は64ビットのアトミック演算だけで、マテリアル評価は決して重複しない。そして、画面に見えている異なるマテリアルの数に応じてフレームごとのオーバーヘッドが増える。

6 ストリーミング、圧縮、そして残された課題

6.1 仮想化ジオメトリ

仮想テクスチャと同様に、Nanite は現在のビューが必要とするものだけを保持する。クラスターグループは固定サイズ（128 KB）のページに詰められる。もっとも粗い段からなる少数のルートページは常駐しているので、どのメッシュも常に何かは描ける。カリング中に、カットが非常駐のクラスタを必要とすると GPU が要求を書き込み、CPU が優先度順にページを読み込み、カットは数フレームかけて細くなる。DAG 構造により、常駐しているより粗い祖先が必ず代役を務められる。

6.2 圧縮

位置はメッシュごとの格子に量子化され、クラスタの境界に対する相対値として、クラスタごとのビット幅で格納される。法線は八面体符号化を用い、インデックスはクラスタ内の頂点集合に対する局所値として格納される。メモリ上の形式は GPU で直接復号でき、ディスク上の形式は汎用圧縮器 (Oodle) がさらに縮められるバイト指向の配置を加える。公表値では、Nanite メッシュはディスク上で入力三角形あたり約14バイトであり、LOD 一式をもつ通常の非 Nanite メッシュより小さい。

6.3 限界とその後の拡張

- **集合的なジオメトリ。** Nanite が簡略化するのは曲面である。草、葉、髪は多数の離れた薄い断片であり、簡略化すると体積がやせ、自己遮蔽によるカリングもきかない。フォリッジ対応は段階的に進んだ (マスクマテリアル、プログラマブルラスタ、そして 5.7 ではインスタンス化したアセンブリ、スキニングによる変形、遠景の樹冠向けのボクセル的表現を加えた実験的なフォリッジ経路)。
- **変形。** 当初は静的メッシュのみだったが、スケルタルメッシュ、World Position Offset、ディスプレイメントによるテッセレーションが 5.x の各版で追加された。
- **カットでの面のちらつき。** LOD はグループ単位で切り替わるため、移動に伴ってトポロジーがわずかに変わることがある。 $\tau \approx 1$ ピクセルでは通常見えないが、刻まれた文字のような鋭い細部はちらつくことがある。
- **影。** 従来のシャドウマップではマイクロジオメトリを解像できないため、Nanite は仮想シャドウマップ (同じ Nanite パイプラインで描かれる $16k^2$ の仮想シャドウマップ) と組み合わせられる。

7 Lumen : 問題設定

グローバルイルミネーションとは、直接光の項だけでなくレンダリング方程式そのものを解くことである。

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} f_r(\mathbf{x}, \omega_i, \omega_o) L_i(\mathbf{x}, \omega_i) (\mathbf{n} \cdot \omega_i) d\omega_i, \quad (3)$$

$$L_i(\mathbf{x}, \omega_i) = L_o(\text{trace}(\mathbf{x}, \omega_i), -\omega_i).$$

L_i の中の再帰が光を反射させる。オフラインレンダラは、ピクセルあたり数百本のレイでこの積分をモンテカルロ推定する。1440p・60fps のゲームが払えるのは、ピクセルあたりおそらく半本のレイであり、しかも相手は完全に動的な世界（焼き込みライトマップなし）で、レイトレーシング用の構造に全詳細に入れるには密すぎる Nanite ジオメトリでできている。

要点

実際のシーンを全詳細でトレースしない。ピクセルごとにもトレースしない。疎なプローブ群からトレースし、ヒット先を事前ライティング済みのサーフェスキャッシュで引ける安価な代理表現（符号付き距離場）に対してトレースする。キャッシュを自分自身に還流させて、時間をかけて無限回の反射を得る。高価なレイは重要な箇所だけに使い、安く正確なものから安く粗いものへと順に試す。

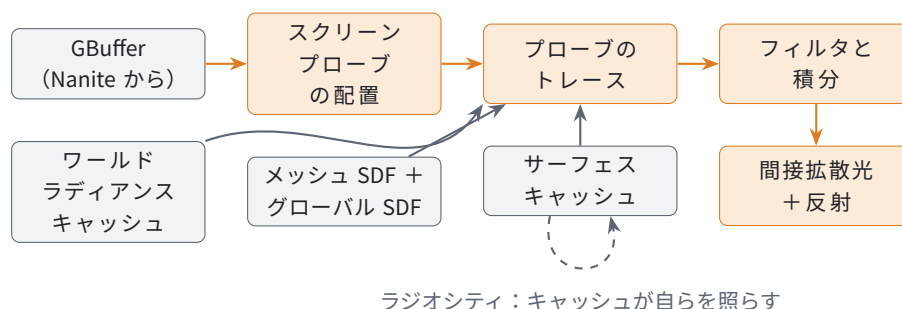


図6 Lumen の全体像。橙は毎フレームのスクリーン空間の処理、灰色はワールド空間のデータ構造で、予算内で少しずつ更新される。

8 レイのためのシーン表現

8.1 メッシュ距離場

各メッシュには事前計算された符号付き距離場（SDF）がある。これは疎な3次元格子で、各テクセルにはもっとも近い曲面までの距離（内部では負）が入っている。SDF のレイマーチングはスフィアトレーシングである。現在の点で保存されている距離は、その半径の内側には何もぶつからないことを保証する半径なので、レイはその分だけ安全に進める。

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \text{SDF}(\mathbf{p}_k) \omega, \quad \text{SDF}(\mathbf{p}_k) < \varepsilon \text{ でヒット.} \quad (4)$$

開けた空間では歩幅が大きく、曲面の近くでは小さくなる。SDF はさらに、ソフトシャドウとコーンの推定をほぼ無料で与える。レイが幾何に最接近した度合い $\min_k \text{SDF}(\mathbf{p}_k)/t_k$ は、レイ周りのコーンがどれだけ遮られているかの近似になるからだ。

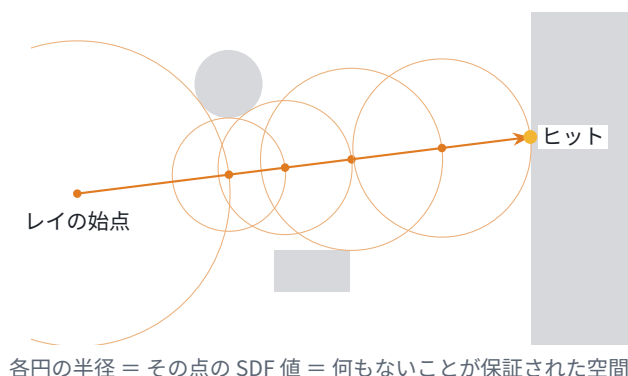


図7 距離場のスフィアトレーシング。空いた空間では大股に進み、曲面の近くで収束する。三角形には一切触れない。

8.2 グローバル距離場

重なり合う多数のメッシュ SDF をレイごとに辿るのは、距離とともに高くつく。Lumen はさらに、すべてのメッシュ SDF を**グローバル距離場**に統合する。これはカメラ中心のクリップマップの集合（外側ほど解像度の粗い入れ子の立方体）で、物体が動くと少しずつ更新される。安価で遠くまで届くが、解像度が低く薄い形状を失う。そのため Lumen は、レイの最初の数メートルには精密なメッシュ SDF を使い、その先でグローバル距離場に切り替える。

8.3 サーフেসキャッシュ

距離場のヒットは、レイがどこで止まったかは教えてくれるが、そこでの光の色は教えてくれない。任意のヒット点でマテリアルの完全なシェーディングと直接光を評価しては遅すぎる。Lumen の答えが**サーフェスキャッシュ**である。

- 各メッシュは少数の**カード**で覆われる。カードはメッシュの周りに置かれた向きをもつ矩形である（概念的には、最大6方向からメッシュを見る正射影カメラのようなもの。複雑なメッシュには構築時の配置処理でより多くのカードが割り当てられる）。
- 各カードは（Nanite で、したがって安価に）アトラスヘラスタライズされ、アルベド、法線、放射、不透明度、深度が格納される。カードの解像度はカメラからの距離に従い、フレームごとの予算内で更新される。
- そのうえでライティングはカード空間で計算される。影付きの直接光に加え、カードのテクセル自体から短いレイを飛ばして集めた間接光である。

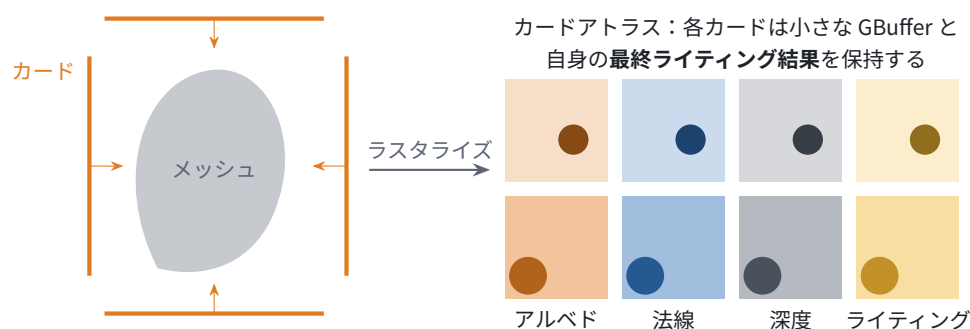


図 8 カードは各メッシュをいくつかの方向からアトラスに写し取る。メッシュ距離場に当たったレイは、そのヒット点をメッシュのカードに投影し、計算済みのライティングを読む。マテリアル評価の代わりにテクスチャフェッチ一回で済む。

8.4 キャッシュ内のラジオシティ：無限反射をただで

サーフェスキャッシュが自身のテクセルの間接光を計算するとき、そのレイはサーフェスキャッシュ自体に当たり、前フレームのライティング結果を読む。1フレームに1回の反射が数フレームで多重反射の光に蓄積される。これは時間方向に回すレンダリング方程式の不動点反復である。

$$L_{\text{cache}}^{(n+1)} = L_{\text{direct}} + \mathcal{T}[L_{\text{cache}}^{(n)}], \quad (5)$$

ここで $\mathcal{T}$ は1回反射の輸送作用素である。物理的なアルベドでは $\mathcal{T}$ のノルムは1未満なので、反復は多重反射の完全な解に収束する。更新は償却される。距離と最近の変化で優先順位を付け、フレームごとに予算内の一部のカードだけを再ライティングする。照明を点けたとき、Lumen の間接光が一拍遅れて目に見えて「落ち着く」のはこのためである。

9 最終ギャザー：スクリーンプローブ

9.1 なぜプローブか

ピクセルあたり1本のレイを飛ばしてデノイズしても、拡散 GI にはノイズが多すぎる。Lumen は、拡散の間接光が低周波であることを利用する。スクリーンプローブを疎に配置し（既定では 16×16 ピクセルのタイルごとに一つ、深度や法線が急に変わるところには追加で配置）、各プローブから半球全体をトレースし、ピクセルごとにプローブ間を補間する。

9.2 プローブが保持するもの

各プローブは、半球にわたる入射ラディアンスを八面体マップ（球を正方形に展開したもの）に格納する。典型的には 8×8 方向である。プローブあたり64本のレイを256ピクセルに一つのプローブで飛ばせば $64/256 = 0.25$ 本／ピクセルだが、各ピクセルは近隣プローブを補間し、プローブは時間方向にも再利用されるので、実質的にははるかに多くの方向について積分していることになる。

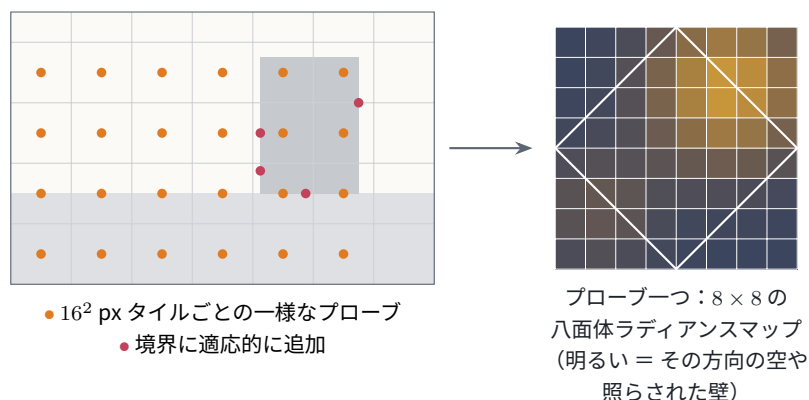


図9 スクリーンプローブは入射光を疎にサンプリングする。各プローブは、周囲の世界の小さな方向別の画像を保持する。

9.3 プローブレイの重点サンプリング

プローブの64本のレイはどこへ向けるべきか。一様な方向では、半球の暗い部分にレイを浪費する。Lumen は、すでに分かっている二つの項からサンプリングの確率密度 (PDF) を作る。プローブを使うピクセルの BRDF と法線、そしてプローブ自身の前フレームのラディアンズ (再投影したもの) である。明るかった方向や BRDF が重く重み付けする方向にはより多くのレイが向かい、その PDF で割ることでモンテカルロ推定は不偏に保たれる。

$$\hat{E}(\mathbf{x}) = \frac{1}{N} \sum_{k=1}^N \frac{L_i(\mathbf{x}, \omega_k) (\mathbf{n} \cdot \omega_k)}{p(\omega_k)}, \quad p(\omega) \propto \underbrace{\cos \theta}_{\text{BRDF}} \times \underbrace{\tilde{L}_{\text{prev}}(\omega)}_{\text{前フレーム}}. \quad (6)$$

これはオフラインで使われる積の重点サンプリングと同じ発想であり、ピクセルごとではなくプローブごとに行うことで費用を抑えている。

9.4 フィルタリングと積分

プローブのラディアンズは近隣のプローブ間で空間的にフィルタリングされ (深度や法線が食い違う近隣、レイが遮られていたはずの近隣は除く)、補間しやすい小さな形式 (拡散には低次の球面調和関数) に変換され、各ピクセルが平面距離による重みで近くのプローブを補間する。最後に時間フィルタが複数フレームにわたって蓄積する。

10 トレースのはしご

各プローブレイは、次第に粗くなるが次第に遠くまで届く手法の列に沿ってトレースされる。各段は前の段があきめたとところから引き継ぐ。

1. スクリーントレース。現在の深度バッファの HZB の中をレイで進む。画面上の何かに当たれば、そこにある前フレームの最終ライティング結果を読む。実際に描画されたジオメトリ (Nanite の詳細を含む) と一致し、粗い代理表現が取りこぼす小さな接触反射も拾えるので、これがもっとも正確な情報源である。

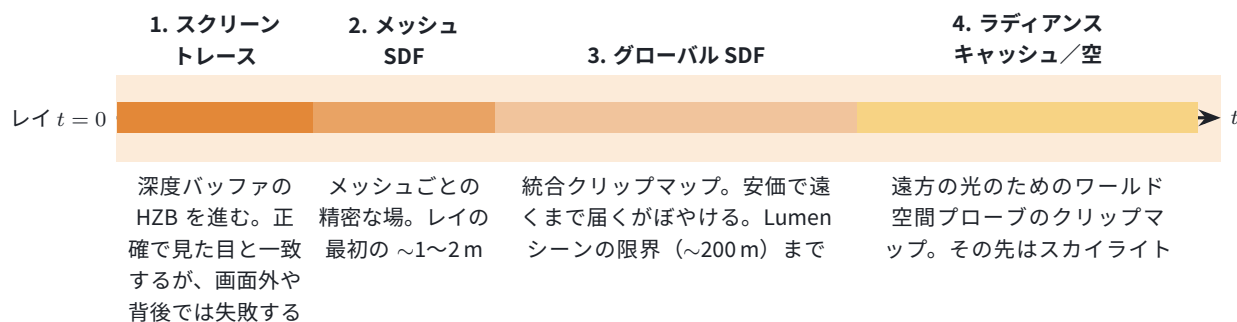


図 10 プローブレイ一本ごとの Lumen のソフトウェアトレースのはしご。第2～3段のヒットはサーフェスキャッシュからライティングを読み、外れたレイは次の段へ落ちる。

2～3. 距離場トレース。 スクリーントレースが画面外に出たり曲面の背後を通ったりしたら、止まったところからメッシュ SDF、次いでグローバル SDF を通して続け、ヒットはサーフェスキャッシュの参照でシェーディングする。

4. ラディアンスキャッシュ。 長いレイは、もっとも高価でありながら一本一本の重要度はもっとも低い。Lumen は第二のワールド空間のプローブ群を保持する。カメラ中心のクリップマップ上の疎なプローブで、角度方向の解像度は高いがゆっくり更新され、すべてのスクリーンプローブから再利用される。一定距離を越えたスクリーンプローブのレイは、そこでトレースをやめてワールドのラディアンスキャッシュを参照する。小さく明るい遠方の特徴（暗い部屋の向こうの明るい窓）がノイズにならず安定するのはこのためである。

10.1 ハードウェアレイトレーシング・モード

RT 対応の GPU では、第2～3段を、ハードウェア BVH を通じた実際の三角形へのレイトレーシングに置き換えられる。Nanite メッシュは簡略化したフォールバックメッシュとして BVH に入る（マイクロポリゴンの全詳細では大きすぎる）が、曲面の近くではスクリーントレースが全詳細を回復する。ヒットではサーフェスキャッシュを読む（速い）か、ヒット点でマテリアルとライティングを完全に評価する（「ヒットライティング」。遅いが、鏡のような反射には必要）かを選べる。ハードウェアモードは薄いジオメトリ、スキンメッシュ、正確な遮蔽をはるかにうまく扱い、ソフトウェアモードはどこでも動き、より多くのインスタンスにスケールする。最近の版（5.6、5.7）では、コンソールでの性能向上に伴い、既定の重心がハードウェア経路へ移りつつある。

11 反射、半透明、その他

反射。 粗い反射はローブの広い間接鏡面反射にすぎないので、スクリーンプローブのラディアンスを再利用する。より滑らかな面（ラフネスがしきい値未満）には、同じはしごを通して低解像度でピクセルごとに専用の反射レイを飛ばし、空間・時間方向にデノイズする。ハードウェアモードの鏡面反射はヒットライティングを使い、反射された物体が粗いサーフェスキャ

ッシュの参照ではなく完全にシェーディングされるようにする。

半透明とフォグ。 低解像度のフロクセル（視錐台に沿ったボクセル）ボリュームを、ボクセル中心から同じ構造へトレースしてライティングする。半透明の面とボリュームメトリックフォグはそれをサンプリングする。

破綻する場所。 SDF の解像度より薄い壁は光が漏れる。カードで覆いきれない形状（深いくぼみ）は間接光が欠けたり暗くなったりする。サーフェスキャッシュと時間フィルタは数フレームかけて収束するので、急な照明変化には遅れが出る。放射面はたまたま当たったレイでしか見つからず直接サンプリングされないため、放射面だけで照らす大きなシーンはノイズが多い。（5.5 で導入され 5.7 で拡張された MegaLights は、多数の光源からの直接光という別の問題を扱う。）

12 二つはどう噛み合うか

	Nanite	Lumen
問題	ジオメトリの費用がコンテンツ量に比例する	ライティングの費用がピクセルあたりのレイ数に比例する
中心の手法	クラスタ DAG と1ピクセル誤差のカット	疎なプローブ、代理シーン、キャッシュ
階層	クラスタ DAG、グループ上のBVH	SDF クリップマップ、プローブのクリップマップ、HZB のミップ
時間方向の再利用	前フレームの可視集合（パス1）	サーフェスキャッシュ、ラディアンスキャッシュ、時間フィルタ、重点サンプリングのPDF
空間方向の再利用	一つのクラスタが多数のピクセルに	一つのプローブが約256ピクセルに
GPU の使い方	常駐型コンピュート、64ビットアトミック	コンピュートでのプローブトレース、非同期更新
取り除くボトルネック	三角形セットアップ、オーバードロー、VRAM	レイ数、ヒット点でのマテリアル評価
典型的なアーティファクト	まれな LOD のトポロジーのちらつき	光漏れ、急な変化の後の遅れ

依存関係は双方向である。Lumen は Nanite を必要とする。カードはメッシュをラスタライズして写し取るものであり、毎フレーム数千枚のカードを写せるのは Nanite がそれを安く描くからだ。Nanite は Lumen（またはそれに類するもの）を必要とする。映画並みの密度のジオメトリも、平板な環境光で照らせばプラスチックの模型に見える。細部を本物らしく見せるのは、Lumen が与える微小な遮蔽と反射光である。

共通する設計原理

どちらも出力感応型のアルゴリズムである。「世界にどれだけあるか」ではなく「画面はどれだけ見せられるか」を問い、そのうえで、その問いを安く答えられるようにする階層、安い答えが危険な方向に決して間違わないようにする保守的な境界、そして今フレームの答えの大半を前フレームの答えにする時間的一貫性を使う。仮想メモリや仮想テクスチャと同じ型を、三角形と光に当てはめたものである。

参考文献 Karis, Stubbe, Wihlidal, A Deep Dive into Nanite Virtualized Geometry, SIGGRAPH 2021 Advances in Real-Time Rendering. Wright, Narkowicz, Kelly, Lumen: Real-time Global Illumination in Unreal Engine 5, SIGGRAPH 2022 Advances. Burns & Hunt, The Visibility Buffer, JCGT 2013. Hart, Sphere Tracing, The Visual Computer 1996. Cigolle ほか, A Survey of Efficient Representations for Independent Unit Vectors（八面体符号化）, JCGT 2014. Epic Games, Unreal Engine 5.7 リリースノート。